

Initial Setup

If you are running this notebook on Google Colab you should:

1. Activate GPU support in Runtime -> Change Runtime Type -> Hardware Accelerator
2. Check if pyCUDA is installed: it will not be, because your Python environment will originally be practically empty (except maybe for `numpy`).

In [33]: `import pycuda`

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
/tmp/ipykernel_450/2872818002.py in <cell line: 0>()  
----> 1 import pycuda
```

```
ModuleNotFoundError: No module named 'pycuda'
```

```
-----  
NOTE: If your import is failing due to a missing package, you can  
manually install dependencies using either !pip or !apt.
```

```
To view examples of installing some common dependencies, click the  
"Open Examples" button below.  
-----
```

In [105... `!pip install pycuda`

```
Requirement already satisfied: pycuda in /usr/local/lib/python3.12/dist-packages (2026.1)  
Requirement already satisfied: pytools>=2011.2 in /usr/local/lib/python3.12/dist-packages (from pycuda) (2026.1.1)  
Requirement already satisfied: platformdirs>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from pycuda) (4.10.0)  
Requirement already satisfied: mako in /usr/lib/python3/dist-packages (from pycuda) (1.1.3)  
Requirement already satisfied: siphash24>=1.6 in /usr/local/lib/python3.12/dist-packages (from pytools>=2011.2->pycuda) (1.8)  
Requirement already satisfied: typing-extensions>=4.5 in /usr/local/lib/python3.12/dist-packages (from pytools>=2011.2->pycuda) (4.16.0)
```

In [106... `import pycuda`

3. Change the notebook name
4. Check which version of pyCUDA is installed

In [107... `!nvcc --version`

```
nvcc: NVIDIA (R) Cuda compiler driver  
Copyright (c) 2005-2025 NVIDIA Corporation  
Built on Fri_Feb_21_20:23:50_PST_2025  
Cuda compilation tools, release 12.8, V12.8.93  
Build cuda_12.8.r12.8/compiler.35583870_0
```

Explore the bash

If you are running this on Colab it might be useful for you to get comfortable with your directory tree. Try to explore your bash using your Jupyter Notebook cells, e.g. using the following instructions. Note that some of those can be given without used "!" : they are called magic instructions.

In [108... `ls`

```
sample_data/  VecAdd*  VecAdd.cu
```

In [109... `mkdir test_dir`

In [110... `cd test_dir`

```
/content/test_dir
```

In [111... `ls`

In [112... `!touch ciao`

In [113... `ls`

```
ciao
```

In [114... `rm ciao`

In [115... `ls`

In [116... `pwd`

```
Out[116...  '/content/test_dir'
```

```
In [117...  cd ..  
/content
```

```
In [118...  !rmdir test_dir
```

```
In [119...  !gcc --version
```

```
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0  
Copyright (C) 2021 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Specifics of the GPU

Let's try to understand the specifics of the GPU we are going to use. Try this instruction:

```
In [120...  !nvidia-smi
```

```
Mon Aug  3 09:55:59 2026  
+-----+  
| NVIDIA-SMI 580.82.07                Driver Version: 580.82.07      CUDA Version: 13.0     |  
+-----+-----+-----+  
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |  
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |  
|                                           | MIG M.         |  
+-----+-----+-----+  
|    0   Tesla T4               Off      | 00000000:00:04:0 Off |                    0 |  
| N/A   63C    P0              29W / 70W | 115MiB / 15360MiB |      0%      Default |  
+-----+-----+-----+  
+-----+  
| Processes: |  
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |  
|       ID    ID                          | Usage      |  
+-----+  
|    0   N/A   N/A           450      C   /usr/bin/python3              112MiB |  
+-----+
```

You can also use `pyCUDA` to inspect the GPU, interrogating the functions of the driver...

```
In [121...  import pycuda.driver as drv  
drv.init()  
drv.get_version()  
devn=drv.Device.count()  
print("N GPU "+str(devn))  
devices = []  
for i in range(devn):  
    devices.append(drv.Device(i))  
for sp in devices:  
    print("GPU name: "+str(sp.name))  
    print("Compute Capability = "+str(sp.compute_capability()))  
    print("Total Memory = "+str(sp.total_memory()/(2.**20))+ ' MBytes')  
    attr = sp.get_attributes()  
    attr = sp.get_attributes()  
# We will print only the first 3 item to shorten the output, but there are much more info here  
print(list(attr.items())[:3])
```

N GPU 1

GPU name: <bound method name of <pycuda._driver.Device object at 0x7fa34b3d0cc0>>

Compute Capability = (7, 5)

Total Memory = 14912.6875 MBytes

[(pycuda._driver.device_attribute.ASYNC_ENGINE_COUNT, 3), (pycuda._driver.device_attribute.CAN_MAP_HOST_MEMORY, 1), (pycuda._driver.device_attribute.CAN_USE_HOST_POINTER_FOR_REGISTERED_MEM, 1)]

... or using the `DeviceData()` method:

```
In [122...  from pycuda.tools import DeviceData  
import pycuda.autoint  
specs = DeviceData()  
print(f"Max threads per block = {specs.max_threads}")  
print(f"Warp size = {specs.warp_size}")  
print(f"Warps per MP = {specs.warps_per_mp}")  
print(f"Thread Blocks per MP = {specs.thread_blocks_per_mp}")  
print(f"Registers = {specs.registers}")  
print(f"Shared memory = {specs.shared_memory}")
```

Max threads per block = 1024
Warp size = 32
Warps per MP = 64
Thread Blocks per MP = 8
Registers = 65536
Shared memory = 49152

Example of GPU usage in C

Let's try and write and compile a GPU program in C. For this example we are gonna use the simple program to add vectors that we have seen before. Note that the instruction at the beginning of the cell is used to save its content in a text file. Use this trick to run C files without exiting the notebook.

```
In [123... %%writefile VecAdd.cu
# include <stdio.h>
# include <cuda_runtime.h>
// CUDA Kernel
__global__ void vectorAdd(const float *A, const float *B, float *C, int numElements)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < numElements)
    {
        C[i] = A[i] + B[i];
    }
}

/**
 * Host main routine
 */
int main(void)
{
    int numElements = 15;
    size_t size = numElements * sizeof(float);
    printf("[Vector addition of %d elements]\n", numElements);

    float a[numElements], b[numElements], c[numElements];
    float *a_gpu, *b_gpu, *c_gpu;

    cudaMalloc((void **)&a_gpu, size);
    cudaMalloc((void **)&b_gpu, size);
    cudaMalloc((void **)&c_gpu, size);

    for (int i=0; i<numElements; ++i ){
        a[i] = i*i;
        b[i] = i;
    }
    // Copy the host input vectors A and B in host memory to the device input vectors in
    // device memory
    printf("Copy input data from the host memory to the CUDA device\n");
    cudaMemcpy(a_gpu, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(b_gpu, b, size, cudaMemcpyHostToDevice);

    // Launch the Vector Add CUDA Kernel
    int threadsPerBlock = 256;
    int blocksPerGrid =(numElements + threadsPerBlock - 1) / threadsPerBlock;
    printf("CUDA kernel launch with %d blocks of %d threads\n", blocksPerGrid, threadsPerBlock);
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>>(a_gpu, b_gpu, c_gpu, numElements);

    // Copy the device result vector in device memory to the host result vector
    // in host memory.
    printf("Copy output data from the CUDA device to the host memory\n");
    cudaMemcpy(c, c_gpu, size, cudaMemcpyDeviceToHost);

    for (int i=0; i<numElements; ++i ){
        printf("%f \n", c[i]);
    }

    // Free device global memory
    cudaFree(a_gpu);
    cudaFree(b_gpu);
    cudaFree(c_gpu);

    printf("Done\n");
    return 0;
}
```

Overwriting VecAdd.cu

```
In [124... ls
```

sample_data/ VecAdd* VecAdd.cu

```
In [125... !nvcc -o VecAdd VecAdd.cu -arch=compute_75 -code=sm_75
```

```
In [126... !./VecAdd
```

```
[Vector addition of 15 elements]
Copy input data from the host memory to the CUDA device
CUDA kernel launch with 1 blocks of 256 threads
Copy output data from the CUDA device to the host memory
0.000000
2.000000
6.000000
12.000000
20.000000
30.000000
42.000000
56.000000
72.000000
90.000000
110.000000
132.000000
156.000000
182.000000
210.000000
Done
```

pyCUDA implementation

Let's navigate our first example of `pyCUDA` by writing a program that does the same thing as the previous `VecAdd` but using Python.

First, import the necessary modules (you'll soon what they are used for).

```
In [127... from pycuda import autoint
from pycuda import gpuarray
import numpy as np
```

Define the vectors `a`, `b` and `c` in the host. They all have a lenght of 15, `a` with numbers from 0 to 14 and `b` with their squares. `c` is initialized to 0.

```
In [128... aux = range(15)
a = np.array(aux).astype(np.float32)
b = (a*a).astype(np.float32)
c = np.zeros(len(aux)).astype(np.float32)
```

Define the vectors on the GPU and copy the content of `a`, `b` and `c` inside the allocated memory space.

```
In [129... a_gpu = gpuarray.to_gpu(a)
b_gpu = gpuarray.to_gpu(b)
c_gpu = gpuarray.to_gpu(c)
```

The first simple method to sum vectors, consists in just using the `+` operator.

```
In [130... c_gpu=a_gpu+b_gpu
c_gpu
```

```
Out[130... array([ 0.,  2.,  6., 12., 20., 30., 42., 56., 72., 90., 110.,
        132., 156., 182., 210.], dtype=float32)
```

Another way to do it is using the `ElementwiseKernel` method from the `pycuda.elementwise` library, which lets us apply the same operation to all vector elements.

```
In [131... from pycuda.elementwise import ElementwiseKernel
myCudaFunc = ElementwiseKernel(arguments = "float *a, float *b, float *c",
                                operation = "c[i] = a[i]+b[i]",
                                name = "mySumK")

myCudaFunc(a_gpu,b_gpu,c_gpu)
c_gpu
```

```
Out[131... array([ 0.,  2.,  6., 12., 20., 30., 42., 56., 72., 90., 110.,
        132., 156., 182., 210.], dtype=float32)
```

The advantage is that we can define even more complex operations than a simple one, such as the one below.

```
In [132... from pycuda.elementwise import ElementwiseKernel
lin_comb = ElementwiseKernel(
    "float a, float *x, float b, float *y, float *z",
    "z[i] = a*x[i] + b*y[i]",
    "linear_combination")
lin_comb(3.,a_gpu,5.,b_gpu,c_gpu)
c_gpu
```

```
Out[132...] array([ 0.,  8., 26., 54., 92., 140., 198., 266., 344.,
        432., 530., 638., 756., 884., 1022.], dtype=float32)
```

The third way is the most generic one: the `SourceModule` method is used to define much more complex kernels. The idea however is that this kernels must be written in Cuda/C. So find the file with the C program written before and follow these next steps:

```
In [133...] from pycuda.compiler import SourceModule
```

- Read the file.

```
In [134...] !ls
```

```
sample_data  VecAdd  VecAdd.cu
```

```
In [135...] cudaCode = open("VecAdd.cu", "r")
myCUDAcode = cudaCode.read()
```

- Compile the code *just-in-time* with the `SourceModule()` method. The warning in the output is just due to the fact that a part of the code that should be run on the host (the `main()` function) was found in the C code. `SourceModule()` just ignores that part, and we could have omitted it.

```
In [136...] myCode = SourceModule(myCUDAcode, options=["-w"])
```

- Now that the kernel is compiled, import it in the Python program.

```
In [137...] importedKernel = myCode.get_function("vectorAdd")
```

- Define the geometry of the GPU.

```
In [138...] nThreadsPerBlock = 256
nBlockPerGrid = 1
nGridsPerBlock = 1
```

- Reset the `c_gpu` vector just to make sure it is empty.

```
In [139...] c_gpu.set(c)
c_gpu
```

```
Out[139...] array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        dtype=float32)
```

Remember: the pointer in the GPU memory is given by the attribute `.gpudata`.

```
In [140...] a_gpu.gpudata
```

```
Out[140...] <pycuda._driver.DeviceAllocation at 0x7fa369fce7a0>
```

```
In [141...] b_gpu.gpudata
```

```
Out[141...] <pycuda._driver.DeviceAllocation at 0x7fa3583d7f40>
```

- Launch the imported kernel with the vectors pointers and the GPU geometry parameters.

```
In [142...] importedKernel(a_gpu.gpudata, b_gpu.gpudata, c_gpu.gpudata, block=(nThreadsPerBlock,nBlockPerGrid,nGridsPerBlock))
c_gpu
```

```
Out[142...] array([ 0.,  2.,  6., 12., 20., 30., 42., 56., 72., 90., 110.,
        132., 156., 182., 210.], dtype=float32)
```

Matrix Sum with `pycuda`

Let's reset our notebook, cleaning the memory used since now.

```
In [143...] %reset
```

Once deleted, variables cannot be recovered. Proceed (y/[n])? y

Now we want to implement a matrix sum using Python. As always, start importing the necessary modules.

```
In [144...] import numpy as np
from pycuda import gpuarray, autoint
import pycuda.driver as cuda
```

```
from pycuda.tools import DeviceData
from pycuda.tools import OccupancyRecord as occupancy
```

Now initialize the matrices (arrays) with the proper dimensions.

```
In [145... presCPU, presGPU = np.float32, 'float'
#presCPU, presGPU = np.float64, 'double'
a_cpu = np.random.random((512,512)).astype(presCPU)
b_cpu = np.random.random((512,512)).astype(presCPU)
c_cpu = np.zeros((512,512), dtype=presCPU)
```

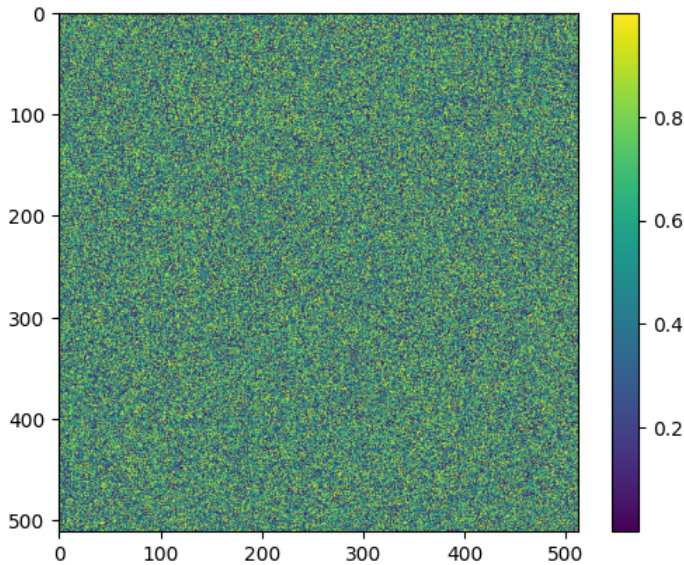
The next line is an instruction to the Python interpreter of the notebook (Ipython) and it is like saying: "Be prepared to use `matplotlib` in the following code cells". This will be useful to easily visualize our matrices directly in the output cells of the notebook.

```
In [146... %matplotlib inline
```

```
In [147... from matplotlib import pyplot as plt
```

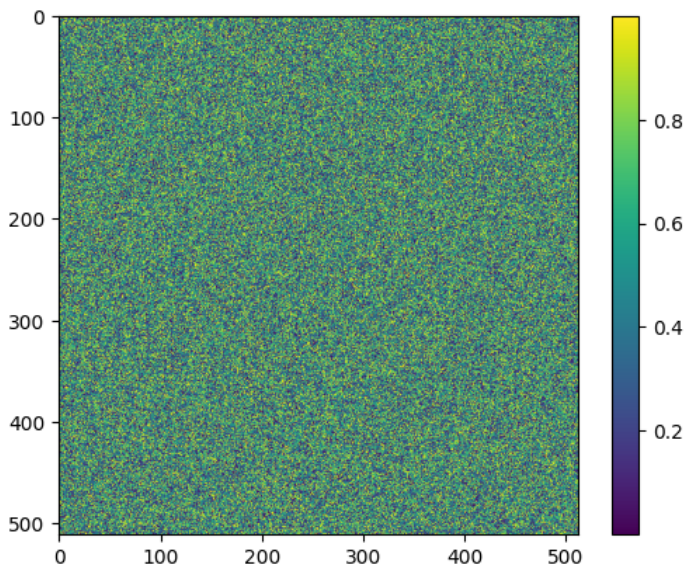
```
In [148... plt.imshow(a_cpu)
plt.colorbar()
```

```
Out[148... <matplotlib.colorbar.Colorbar at 0x7fa358357710>
```



```
In [149... plt.imshow(b_cpu)
plt.colorbar()
```

```
Out[149... <matplotlib.colorbar.Colorbar at 0x7fa3581c2450>
```



Copy the arrays on the GPU.

```
In [150... a_gpu = gpuarray.to_gpu(a_cpu)
b_gpu = gpuarray.to_gpu(b_cpu)
```

```
c_gpu = gpuarray.to_gpu(c_cpu)
c_gpu
```

```
Out[150...] array([[0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      ...,
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.]], dtype=float32)
```

As a reference, let's compute the sum using the CPU...

```
In [151...] c_cpu=a_cpu+b_cpu
c_cpu
```

```
Out[151...] array([[0.964806 , 0.8630216 , 1.423166 , ..., 0.9747033 , 0.8722979 ,
      0.8727901 ],
      [1.2896549 , 0.85598963, 1.4954324 , ..., 0.9515791 , 1.494953 ,
      0.06189794],
      [0.85879874, 1.745034 , 1.3511494 , ..., 0.628626 , 1.4831268 ,
      0.8735285 ],
      ...,
      [1.3014463 , 1.499606 , 0.9507327 , ..., 1.262653 , 0.95367455,
      0.1705497 ],
      [0.6491481 , 1.3951033 , 0.84906363, ..., 0.83346176, 1.0688338 ,
      0.9406847 ],
      [1.5484376 , 0.30774814, 1.8046453 , ..., 1.6575204 , 1.0121435 ,
      1.2504239 ]], dtype=float32)
```

... and measure the execution time using the `%timeit` instruction.

```
In [152...] t_cpu = %timeit -o c_cpu = a_cpu+b_cpu
```

108 µs ± 1.49 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)

Define the kernel that will execute the sum on the device.

```
In [153...] cudaKernel = '''
__global__ void matrixAdd(float *A, float *B, float *C)
{
    int tid_x = blockDim.x * blockIdx.x + threadIdx.x;
    int tid_y = blockDim.y * blockIdx.y + threadIdx.y;
    int tid   = gridDim.x * blockDim.x * tid_y + tid_x;
    C[tid] = A[tid] + B[tid];
}
'''
```

Just like before, compile this kernel and generate the function to use in Python.

```
In [154...] from pycuda.compiler import SourceModule
myCode = SourceModule(cudaKernel)
addMatrix = myCode.get_function("matrixAdd") # The output of get_function is the GPU-compiled function.
```

```
In [155...] type(addMatrix)
```

```
Out[155...] pycuda_driver.Function
def function_call(func, *args, **kwargs)

<no docstring>
```

Now we should define the GPU geometry in a clever way. For example we can try to exploit every threads inside a block. How many are there?

```
In [156...] dev = cuda.Device(0) #
devdata = DeviceData(dev)
print(f"Using device : {dev.name()}")
print(f"Max threads per block: {dev.max_threads_per_multiprocessor}")
```

Using device : Tesla T4
Max threads per block: 1024

The maximum dimension of the blocks is 32x32. Since we are working with 512x512 matrices we can use 16x16 grids.

```
In [157...] cuBlock = (32,32,1)
cuGrid = (16,16,1)
```

We have already compiled the kernel with `SourceModule`. We have two ways to launch it:

- calling the function directly (just like we did for the vector sum):

```
kernelFunction(arg1,arg2, ... ,block=(n,m,l),grid=(r,s,t)
```

- using the *preparation*

```
kernelFunction.prepare('ABC..') # Each letter corresponds to an input data type of the function, i =  
int, f = float, P = pointer, ...  
kernelFunction.prepared_call(grid,block,arg1.gpudata,arg2,...) # When using GPU arrays, they should be  
passed as pointers with the attribute 'gpudata'
```

In our case, the first method reads

```
In [158... addMatrix(a_gpu,b_gpu,c_gpu,block=cuBlock,grid=cuGrid)
```

However, with the preparation we can obtain better performances and measure the execution time.

```
In [159... addMatrix.prepare('PPP')  
addMatrix.prepared_call(cuGrid,cuBlock,a_gpu.gpudata,b_gpu.gpudata,c_gpu.gpudata)  
time2 = addMatrix.prepared_timed_call(cuGrid,cuBlock,a_gpu.gpudata,b_gpu.gpudata,c_gpu.gpudata)  
time2()
```

```
Out[159... 2.1919999271631243e-05
```

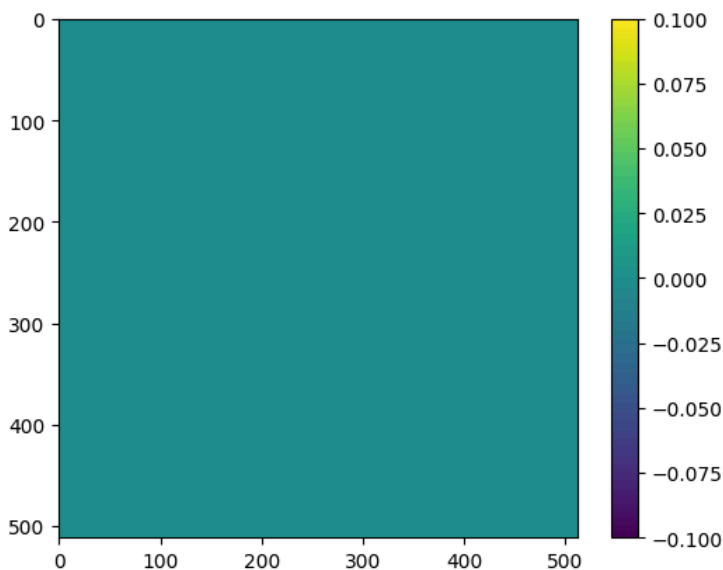
Copy the result from the GPU to CPU to check it.

```
In [160... c = c_gpu.get()
```

Take a look at the comparison between the host result and the device one using the plot of their difference.

```
In [161... plt.imshow(c-c_cpu, interpolation='none')  
plt.colorbar()
```

```
Out[161... <matplotlib.colorbar.Colorbar at 0x7fa35824aa80>
```



```
In [162... np.sum(np.sum(np.abs(c_cpu-c)))
```

```
Out[162... np.float32(0.0)
```

You can clearly see that the results are the same!

Matrix multiplication

Now that you have become somewhat practical with `pycuda`, you should be able to understand how matrix multiplication is implemented in the following example.

Here is a kernel for matrix multiplication:

```
In [163... cudaKernel12 = '''  
__global__ void matrixMul(float *A, float *B, float *C)  
{  
    int tid_x = blockDim.x * blockIdx.x + threadIdx.x; // Row  
    int tid_y = blockDim.y * blockIdx.y + threadIdx.y; // Column  
    int matrixDim = gridDim.x * blockDim.x;  
    int tid = matrixDim * tid_y + tid_x; // element i,j
```

```

float aux=0.0f;

for ( int i=0 ; i<matrixDim ; i++ ){
    //
    aux += A[matrixDim * tid_y + i]*B[matrixDim * i + tid_x] ;

}

C[tid] = aux;

}
...

```

Compile it and import it with `SourceModule` .

```

In [164...] myCode = SourceModule(cudaKernel2)
mulMatrix = myCode.get_function("matrixMul")

```

Execute it with the same block structure defined for the matrix sum.

```

In [165...] mulMatrix(a_gpu,b_gpu,c_gpu,block=cuBlock,grid=cuGrid)

```

Use the CPU as reference.

```

In [166...] dotAB = np.dot(a_cpu, b_cpu)

```

Now, verify if the CPU and GPU results are equal.

```

In [167...] diff = np.abs(c_gpu.get()-dotAB)
np.sum(diff)

```

```

Out[167...] np.float32(8.632927)

```

```

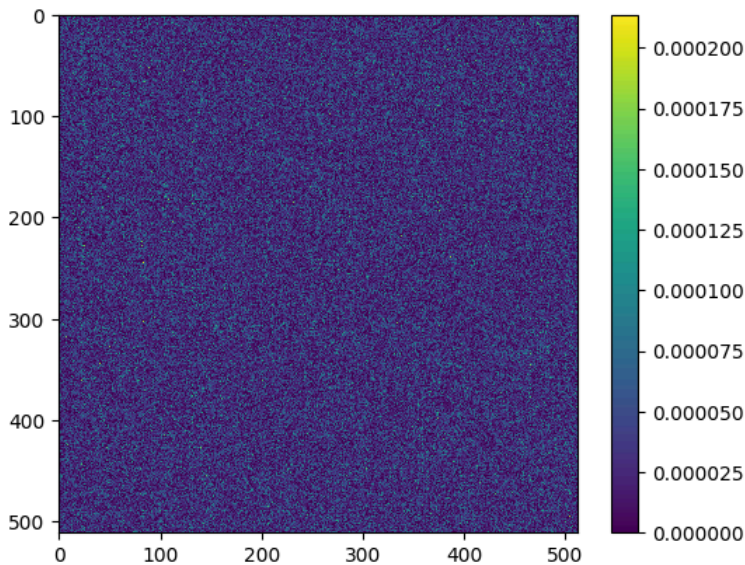
In [168...] plt.imshow(diff,interpolation='none')
plt.colorbar()

```

```

Out[168...] <matplotlib.colorbar.Colorbar at 0x7fa35824a360>

```



As you can see there is a small difference between the two, and the reason is that complex operations such as matrix multiplication can be vulnerable to different numerical and computational errors depending on the processor. One way to reduce this is using higher precision floats for our matrices elements. Here's an example:

```

In [169...] presCPU, presGPU = np.float64, 'double'
a_cpu = np.random.random((512,512)).astype(presCPU)
b_cpu = np.random.random((512,512)).astype(presCPU)
c_cpu = np.zeros((512,512), dtype=presCPU)

a_gpu = gpuarray.to_gpu(a_cpu)
b_gpu = gpuarray.to_gpu(b_cpu)
c_gpu = gpuarray.to_gpu(c_cpu)

a_cpu.dtype

```

```

Out[169...] dtype('float64')

```

```

In [170...] cudaKernel3 = '''
__global__ void matrixMul64(double *A, double *B, double *C)

```

```
{
    int tid_x = blockDim.x * blockIdx.x + threadIdx.x; // Row
    int tid_y = blockDim.y * blockIdx.y + threadIdx.y; // Column
    int matrixDim = gridDim.x * blockDim.x;
    int tid = matrixDim * tid_y + tid_x; // element i,j

    double aux = 0.0;
    for ( int i=0 ; i<matrixDim ; i++){
        //
        aux += A[matrixDim * tid_y + i]*B[matrixDim * i + tid_x] ;

    }

    C[tid] = aux;

}
...
```

```
In [171...] myCode64 = SourceModule(cudaKernel3)
mulMatrix64 = myCode64.get_function("matrixMul64")
mulMatrix64(a_gpu,b_gpu,c_gpu,block=cuBlock,grid=cuGrid)
c_gpu.dtype
```

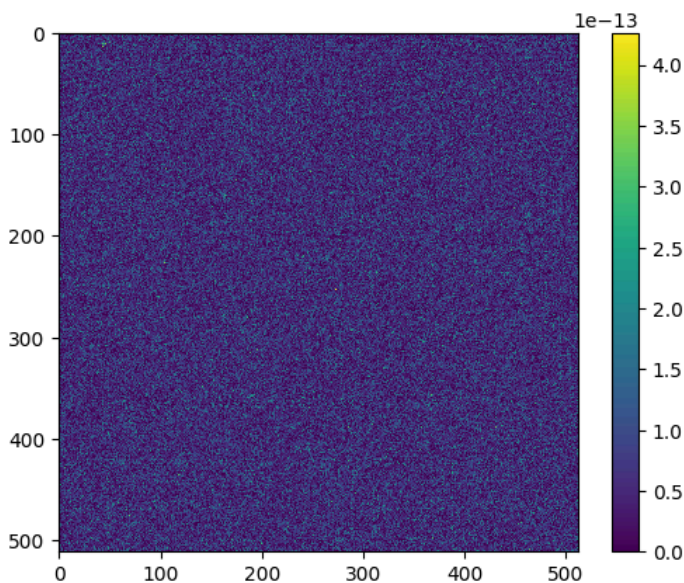
```
Out[171...] dtype('float64')
```

```
In [172...] dotAB = np.dot(a_cpu, b_cpu)
dotAB.dtype
```

```
Out[172...] dtype('float64')
```

```
In [173...] diff = np.abs(c_gpu.get()-dotAB)
plt.imshow(diff,interpolation='none')
plt.colorbar()
```

```
Out[173...] <matplotlib.colorbar.Colorbar at 0x7fa34b38f560>
```



More on vector sum

```
In [174...] %reset
```

Once deleted, variables cannot be recovered. Proceed (y/[n])? y

It is interesting to compare the CPU and GPU execution times for vector addition across varying dimensions.

```
In [175...] %matplotlib inline
```

```
import numpy as np
from matplotlib import pyplot as plt
from time import time
```

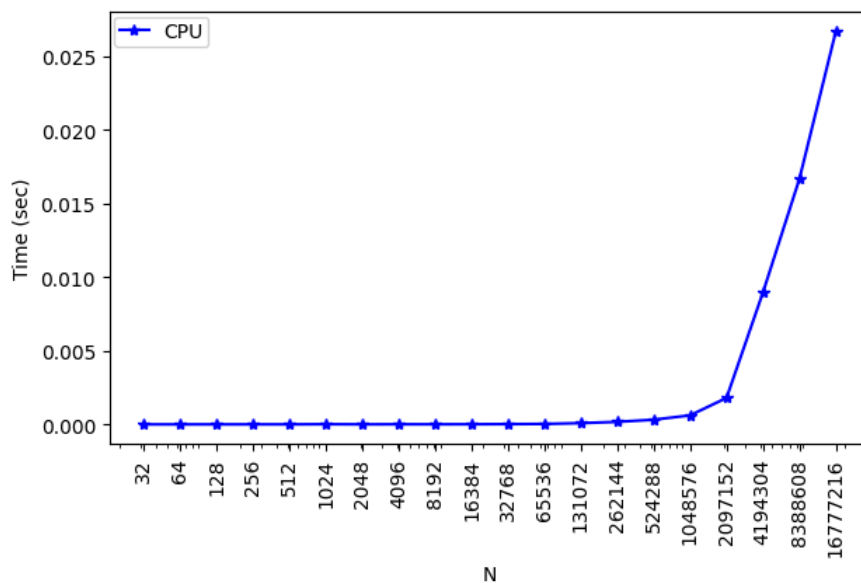
```
In [176...] def myColorRand():
    return (np.random.random(), np.random.random(), np.random.random())
dimension = [2**i for i in range(5,25) ]
myPrec = np.float32
print(dimension)
```

```
[32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144, 524288, 1048576, 2097152, 4194304, 8388608, 16777216]
```

This next piece of code is not very "Pythonic" and the `for` loop can be omitted. However it useful to show the CPU performance, so chill, hakuna matata.

```
In [177... nLoops = 100
timeCPU = []
for n in dimension:
    v1_cpu = np.random.random(n).astype(myPrec)
    v2_cpu = np.random.random(n).astype(myPrec)
    tMean = 0
    for i in range(nLoops):
        t = time()
        v = v1_cpu+v2_cpu
        t = time() - t
        tMean += t/nLoops
    timeCPU.append(tMean)
```

```
In [178... plt.figure(1,figsize=(7,4))
plt.semilogx(dimension, timeCPU, 'b-*', label='CPU')
plt.ylabel('Time (sec)')
plt.xlabel('N')
plt.xticks(dimension, dimension, rotation='vertical')
plt.legend(loc='upper left', labelspace=0.5, fancybox=True, handlelength=1.5, borderaxespad=0.25, borderpad=0.25)
plt.show()
```



We are going to do the same thing on the GPU using the 3 methods explained before:

- Method 1: Simple GPU operations

```
In [179... import pycuda
from pycuda import gpuarray

timeGPU1 = []
bandWidth1 = []

for n in dimension:
    v1_cpu = np.random.random(n).astype(myPrec)
    v2_cpu = np.random.random(n).astype(myPrec)
    t1Mean = 0
    t2Mean = 0

    for i in range(nLoops):
        t = time()
        vaux = gpuarray.to_gpu(v1_cpu)
        t = time() - t
        t1Mean += t/nLoops

    bandwidth1.append(t1Mean)
    v1_gpu = gpuarray.to_gpu(v1_cpu)
    v2_gpu = gpuarray.to_gpu(v2_cpu)

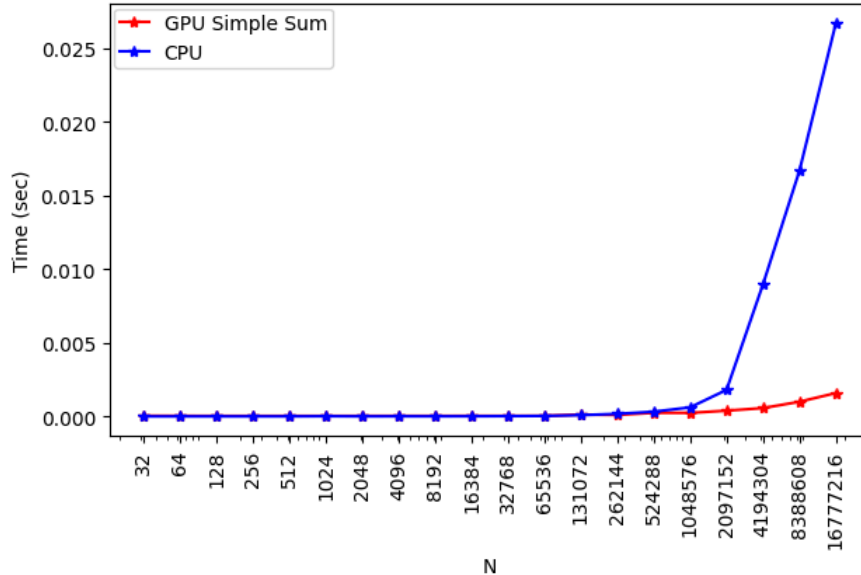
    for i in range(nLoops):
        t = time()
        v = v1_gpu+v2_gpu
        t = time() - t
        t2Mean += t/nLoops

    timeGPU1.append(t2Mean)
```

```
v1_gpu.gpudata.free()
v2_gpu.gpudata.free()
v.gpudata.free()
```

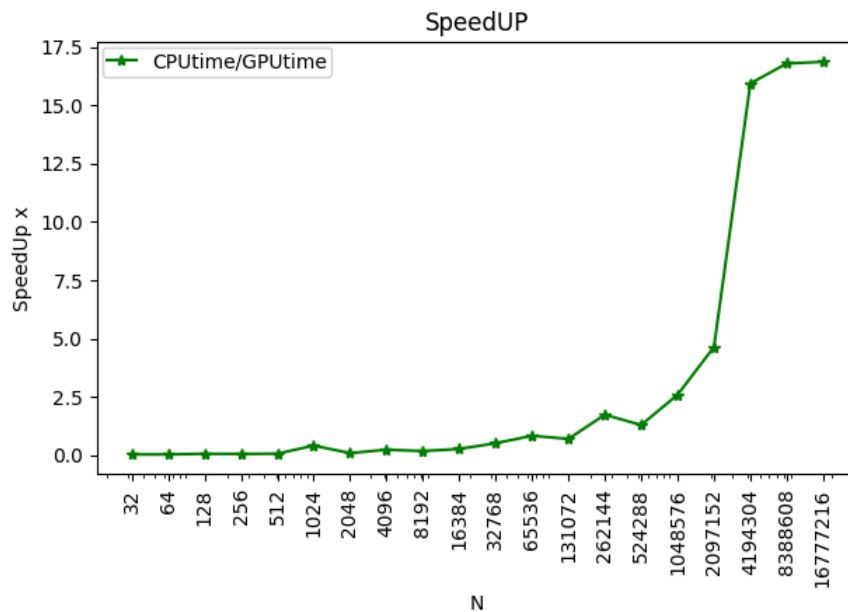
In [180...

```
plt.figure(1,figsize=(7,4))
plt.semilogx(dimension,timeGPU1,'r-*',label='GPU Simple Sum')
plt.semilogx(dimension,timeCPU,'b-*',label='CPU')
plt.ylabel('Time (sec)')
plt.xlabel('N')
plt.xticks(dimension, dimension, rotation='vertical')
plt.legend(loc='upper left', labelspace=0.5, fancybox=True, handlelength=1.5, borderaxespad=0.25, borderpad=0.25)
plt.show()
```



In [181...

```
plt.figure(1,figsize=(7,4))
a = np.array(timeGPU1)
b = np.array(timeCPU)
plt.semilogx(dimension,b/a,'g-*',label='CPUtime/GPUtime')
plt.ylabel('SpeedUp x')
plt.xlabel('N')
plt.title('SpeedUP')
plt.xticks(dimension, dimension, rotation='vertical')
plt.legend(loc='upper left', labelspace=0.5, fancybox=True, handlelength=1.5, borderaxespad=0.25, borderpad=0.25)
plt.show()
```

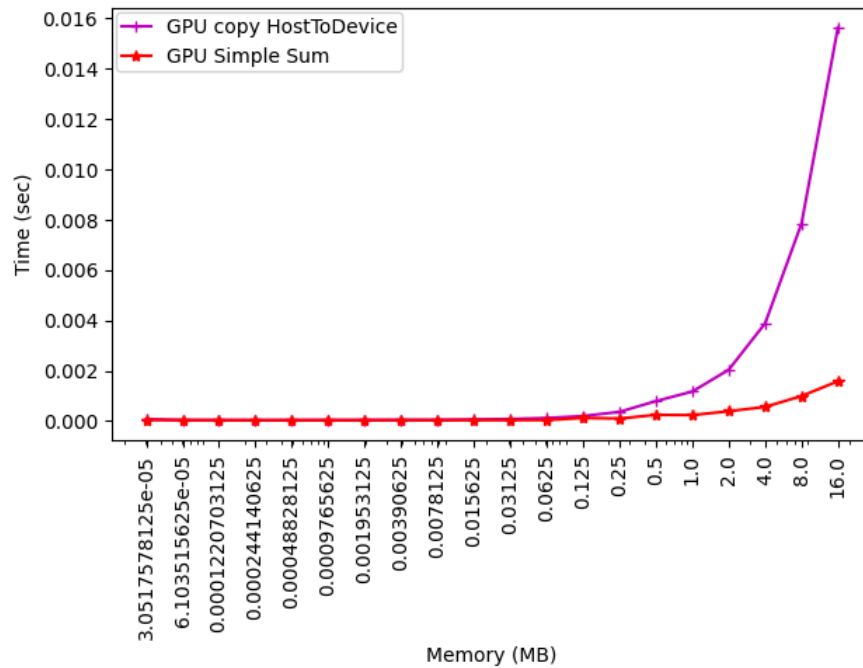


What is the GPU transfer time?

In [182...

```
plt.figure(1,figsize=(7,4))
sizeMB = np.array(dimension)/(2.**20)
plt.semilogx(sizeMB, bandwidth1,'m-+',label='GPU copy HostToDevice')
plt.semilogx(sizeMB, timeGPU1,'r-*',label='GPU Simple Sum')
plt.ylabel('Time (sec)')
plt.xlabel('Memory (MB)')
```

```
plt.xticks(sizeMB, sizeMB, rotation='vertical')
plt.legend(loc='upper left', labelspace=0.5, fancybox=True, handlelength=1.5, borderaxespad=0.25, borderpad=0.25)
plt.show()
```



As expected, the GPU speed up is noticeable only with vectors with high dimensions even if a lot of time is lost to transfer the data from host to device.

- Method 2: `elementwise`

```
In [183... import pycuda.driver as drv
from pycuda.elementwise import ElementwiseKernel
myCudaFunc = ElementwiseKernel(arguments = "float *a, float *b, float *c",
                                operation = "c[i] = a[i]+b[i]",
                                name = "mySumK")

start = drv.Event()
end = drv.Event()
```

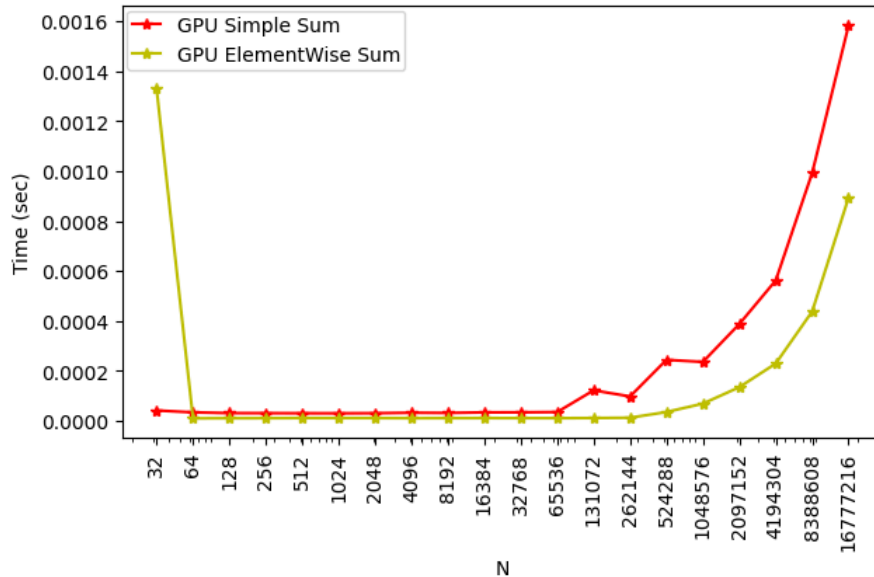
```
In [184... timeGPU2 = []
for n in dimension:
    v1_cpu = np.random.random(n).astype(myPrec)
    v2_cpu = np.random.random(n).astype(myPrec)
    v1_gpu = gpuarray.to_gpu(v1_cpu)
    v2_gpu = gpuarray.to_gpu(v2_cpu)
    vr_gpu = gpuarray.to_gpu(v2_cpu)
    t3Mean=0

    for i in range(nLoops):
        start.record()
        myCudaFunc(v1_gpu,v2_gpu,vr_gpu)
        end.record()
        end.synchronize()
        secs = start.time_till(end)*1e-3
        t3Mean+=secs/nLoops

    timeGPU2.append(t3Mean)
    v1_gpu.gpudata.free()
    v2_gpu.gpudata.free()
    vr_gpu.gpudata.free()
```

```
In [185... plt.figure(1,figsize=(7,4))
plt.semilogx(dimension,timeGPU1,'r-+',label='GPU Simple Sum')
plt.semilogx(dimension,timeGPU2,'y-+',label='GPU Elementwise Sum')
plt.ylabel('Time (sec)')
plt.xlabel('N')
plt.xticks(dimension, dimension, rotation='vertical')
plt.legend(loc='upper left', labelspace=0.5, fancybox=True, handlelength=1.5, borderaxespad=0.25, borderpad=0.25)
```

```
Out[185... <matplotlib.legend.Legend at 0x7fa3583b5430>
```



- Method 3: SourceModule

```
In [186... from pycuda.compiler import SourceModule

presCPU, presGPU = np.float32, 'float'
cudaCode = open("VecAdd.cu", "r")
cudaCode = cudaCode.read()
cudaCode = cudaCode.replace('float', presGPU)
myCode = SourceModule(cudaCode)
vectorAddKernel = myCode.get_function("vectorAdd")
vectorAddKernel.prepare('PPP')
```

```
/tmp/ipykernel_450/1446309357.py:7: UserWarning: The CUDA compiler succeeded, but said the following:
kernel.cu(17): warning #2949-D: function "main" cannot be declared in a linkage-specification
    int main(void)
      ^
```

Remark: The warnings can be suppressed with "-diag-suppress <warning-number>"

```
myCode = SourceModule(cudaCode)
```

```
Out[186... <pycuda._driver.Function at 0x7fa34b136340>
```

```
In [187... timeGPU3 = []
occupancyMeasure=[]
for nt in [32,64,128,256,512,1024]:
    aux = []
    auxOcc = []

    for n in dimension:
        v1_cpu = np.random.random(n).astype(myPrec)
        v2_cpu = np.random.random(n).astype(myPrec)
        v1_gpu = gpuarray.to_gpu(v1_cpu)
        v2_gpu = gpuarray.to_gpu(v2_cpu)
        vr_gpu = gpuarray.to_gpu(v2_cpu)
        cudaBlock = (nt,1,1)
        cudaGrid = (int((n+nt-1)/nt),1,1)

        cudaCode = open("VecAdd.cu", "r")
        cudaCode = cudaCode.read()
        cudaCode = cudaCode.replace('float', presGPU)
        downVar = ['blockDim.x', 'blockDim.y', 'blockDim.z', 'gridDim.x', 'gridDim.y', 'gridDim.z']
        upVar = [str(cudaBlock[0]), str(cudaBlock[1]), str(cudaBlock[2]),
                 str(cudaGrid[0]), str(cudaGrid[1]), str(cudaGrid[2])]
        dicVarOptim = dict(zip(downVar, upVar))

        for i in downVar:
            cudaCode = cudaCode.replace(i, dicVarOptim[i])
        #print cudaCode
        myCode = SourceModule(cudaCode, options=["-w"])
        vectorAddKernel = myCode.get_function("vectorAdd")
        vectorAddKernel.prepare('PPP')

        # Uncomment these lines if you want to see how your code is running
        #print ('Size= ' +str(n)+ " threadsPerBlock= " +str(nt))
        #print (str(cudaBlock)+ " " +str(cudaGrid))

    t5Mean = 0
```

```

for i in range(nLoops):
    timeAux = vectorAddKernel.prepared_timed_call(cudaGrid,cudaBlock,v1_gpu.gpudata,v2_gpu.gpudata,vr_gpu.gpudata)
    t5Mean += timeAux()/nLoops

    aux.append(t5Mean)
    v1_gpu.gpudata.free()
    v2_gpu.gpudata.free()
    vr_gpu.gpudata.free()

timeGPU3.append(aux)
occupancyMesure.append(auxOcc)

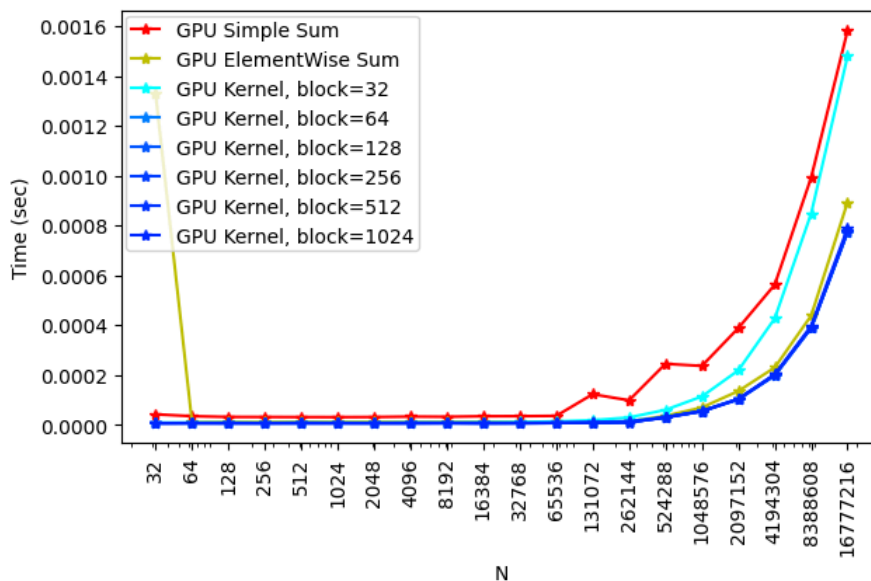
```

```

In [188... plt.figure(1,figsize=(7,4), dpi=100)
plt.semilogx(dimension,timeGPU1,'r-*',label='GPU Simple Sum')
plt.semilogx(dimension,timeGPU2,'y-*',label='GPU ElementWise Sum')
count = 0
for nt in [32,64,128,256,512,1024]:
    plt.semilogx(dimension,timeGPU3[count],'-*',label='GPU Kernel, block={0}'.format(nt),color=(0,1./(count+1),1))
    count+=1
plt.ylabel('Time (sec)')
plt.xlabel('N')
plt.xticks(dimension, dimension, rotation='vertical')
plt.legend(loc='upper left', labelspace=0.5, fancybox=True, handlelength=1.5, borderaxespad=0.25, borderpad=0.25)

```

Out[188... <matplotlib.legend.Legend at 0x7fa34b2e1310>



As you can see, the `SourceModule` method seems to be the fastest, even though a vector sum is too simple to strongly underline the performance difference between different GPU implementations.

Numba

In [189... `%reset`

Once deleted, variables cannot be recovered. Proceed (y/[n])? y

We said that `numpy` exploits the fact that many functions (`ufunc`, universal functions) are compiled in C and act on vector elements autonomously. Thanks to that, `numpy` is used to avoid the GIL mechanism and sped up code execution.

Here we used `numpy` to compute the square root of each element in an array of 10 million elements and compared the execution time with the classic `for` loop:

```

In [190... import numpy as np
import math
x = np.arange(int(1e7), dtype=np.float32)
%timeit np.sqrt(x)
%timeit [math.sqrt(xx) for xx in x]

```

10.1 ms ± 89.1 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
2.07 s ± 901 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

`numba` is a powerful Python module that can be used to vectorize user functions and work with the GPU. The `@vectorize` decorator tells the interpreter to write the function in C and run it in threads (if possible), i.e. avoid the GIL.

```

In [191... import math
import numpy as np
from numba import vectorize

```

```
@vectorize
def cpu_sqrt(x):
    return math.sqrt(x)

%timeit cpu_sqrt(x)
```

10.2 ms ± 859 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)

As you can see now the execution time is comparable with `numpy`.

In the following cells we will work with the GPU using `numba`. We should rewrite the same function as before, but this time the syntax is a little different in order to use the GPU for the execution. It is necessary to specify the input and the output in the decorator with `@vectorize([type_output(type_input)])` (pay attention to the type of your variables!).

```
In [193...] @vectorize(['float32(float32)'], target='cuda')
def gpu_sqrt(x):
    return math.sqrt(x)

%timeit gpu_sqrt(x)
```

27.4 ms ± 4.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

It appears that the GPU version is slower than the CPU one. This is because the operation we are performing is too simple; therefore, the computational gain does not outweigh the overhead of transferring the array to the device.

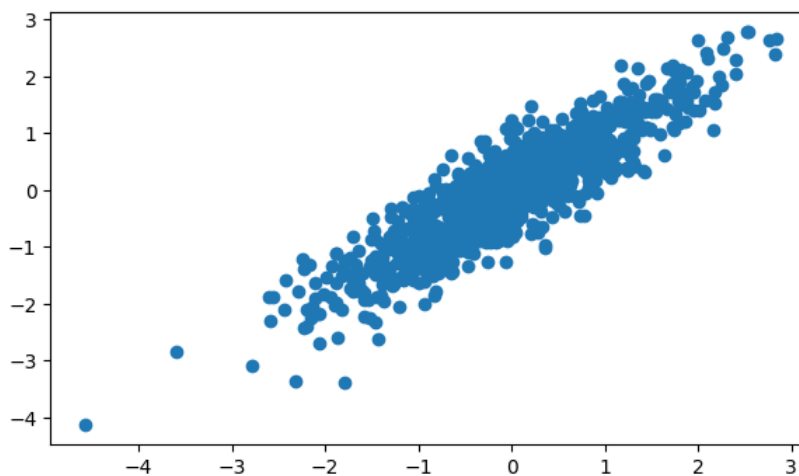
So let's try with a more complex example: we will generate some points in a 2D space with a certain correlation between them and then we will transform their coordinates from cartesian to polar using `numpy` and the GPU.

- `numpy` :

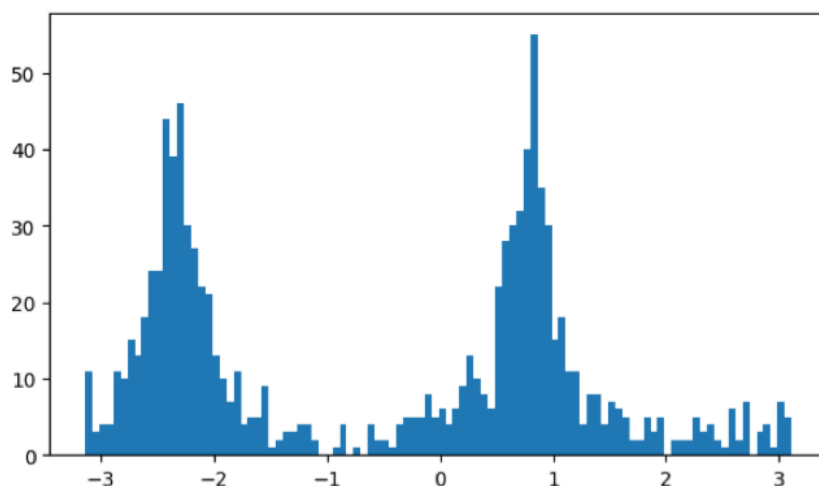
```
In [194...] import matplotlib.pyplot as plt

points = np.random.multivariate_normal([0,0], [[1.,0.9], [0.9,1.]], 1000).astype(np.float32)
plt.figure(figsize=(7, 4), dpi=100)
plt.scatter(points[:,0], points[:,1])
```

Out[194...] <matplotlib.collections.PathCollection at 0x7fa2febe50a0>



```
In [195...] # Polar coordinates
theta = np.arctan2(points[:,1], points[:,0])
plt.figure(figsize=(7, 4), dpi=100)
_ = plt.hist(theta, bins=100)
```



In the histogram two peaks emerge because of the correlation.

- GPU:

```
In [196... @vectorize(['float32(float32, float32)'], target='cuda')
def gpu_arctan2(y, x):
    theta = math.atan2(y,x)
    return theta
```

```
In [198... theta = gpu_arctan2(points[:,1], points[:,0])
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_450/2890360530.py in <cell line: 0>()
----> 1 theta = gpu_arctan2(points[:,1], points[:,0])

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/vectorizers.py in __call__(self, *args, **kws)
    32         the input arguments.
    33         """
--> 34         return CUDAUFuncMechanism.call(self.functions, args, kws)
    35
    36     def reduce(self, arg, stream=0):

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/deviceufunc.py in call(cls, typemap, args, kws)
    362         any_device = True
    363         else:
--> 364             dev_a = cr.to_device(a, stream=stream)
    365             devargs.append(dev_a)
    366

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/vectorizers.py in to_device(self, hostary, stream)
    174
    175     def to_device(self, hostary, stream):
--> 176         return cuda.to_device(hostary, stream=stream)
    177
    178     def to_host(self, devary, stream):

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/cudadrv/devices.py in _require_cuda_context(*args, **kws)
    231     def _require_cuda_context(*args, **kws):
    232         with _runtime.ensure_context():
--> 233             return fn(*args, **kws)
    234
    235     return _require_cuda_context

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/api.py in to_device(obj, stream, copy, to)
    138     """
    139     if to is None:
--> 140         to, new = devicearray.auto_device(
    141             obj, stream=stream, copy=copy, user_explicit=True
    142         )

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/cudadrv/devicearray.py in auto_device(obj, stream, copy, user_explicit)
    917         obj, copy=False if numpy_version < (2, 0) else None, subok=True
    918     )
--> 919     sentry_contiguous(obj)
    920     devobj = from_array_like(obj, stream=stream)
    921     if copy:

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/cudadrv/devicearray.py in sentry_contiguous(ary)
    888     core = array_core(ary)
    889     if not core.flags["C_CONTIGUOUS"] and not core.flags["F_CONTIGUOUS"]:
--> 890         raise ValueError(errmsg_contiguous_buffer)
    891
    892

ValueError: Array contains non-contiguous buffer and cannot be transferred as a single memory region. Please ensure contiguous buffer with numpy .ascontiguousarray()
```

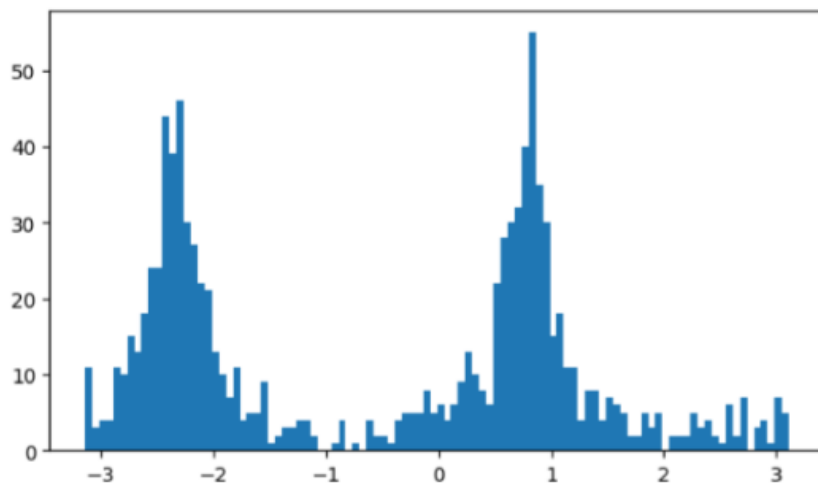
18

This won't work since the slices we extracted are not contiguous in memory, and contiguous arrays must be passed as inputs. Luckily, there's a specific function to make them contiguous:

```
In [199... x = np.ascontiguousarray(points[:,0])
y = np.ascontiguousarray(points[:,1])

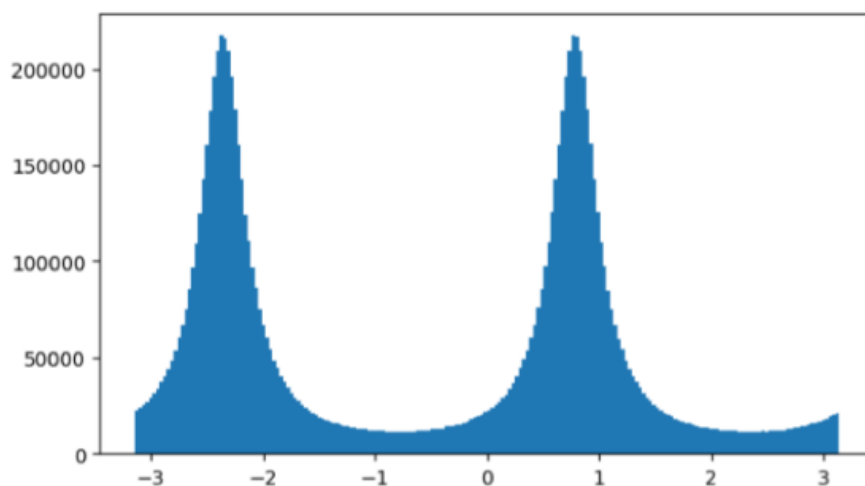
theta = gpu_arctan2(y, x)
plt.figure(figsize=(7, 4), dpi=100)
_ = plt.hist(theta, bins=100)
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely result in GPU under-utilization due to low occupancy.
warn(errors.NumbaPerformanceWarning(msg))
```



Now that we know how everything works, let's generate 10 million points, run the `numpy` and GPU (and even the plain Python implementation) functions and look at the difference in execution time.

```
In [200...] points = np.random.multivariate_normal([0,0], [[1.,0.9], [0.9,1.]], int(1e7)).astype(np.float32)
x = np.ascontiguousarray(points[:,0])
y = np.ascontiguousarray(points[:,1])
plt.figure(figsize=(7, 4), dpi=100)
_ = plt.hist(gpu_arctan2(y, x), bins=200)
```



```
In [201...] %timeit np.arctan2(y, x)
```

21.4 ms ± 2.18 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)

```
In [202...] %timeit gpu_arctan2(y, x)
```

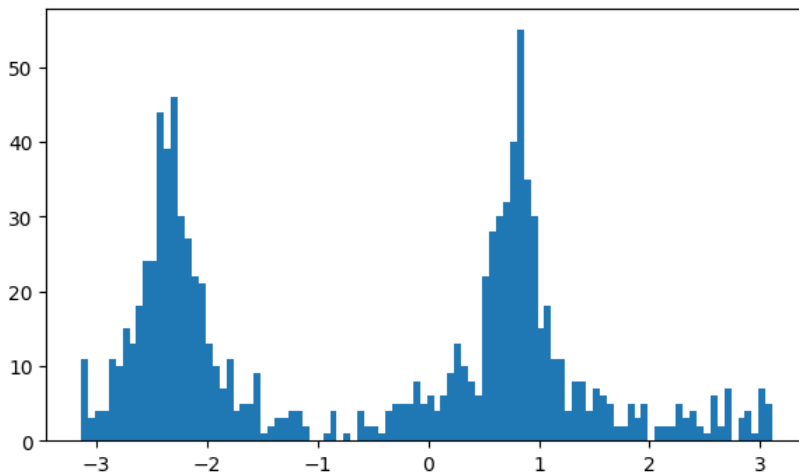
44.3 ms ± 2.28 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

```
In [203...] # Plain Python
%timeit [math.atan2(point[1], point[0]) for point in points]
```

5.33 s ± 755 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

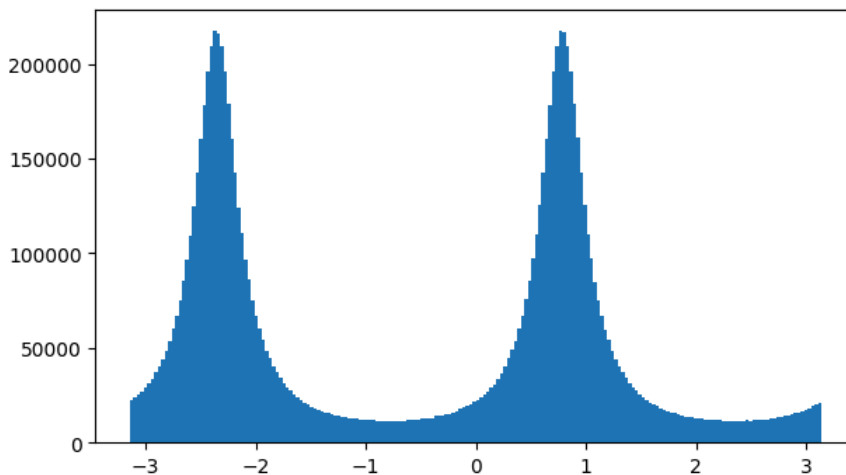
Note: LOL, this example was meant to show that the GPU is faster for more complex computations, but I think that Prof. Lamanna was maybe using a different GPU when he wrote this code, since the `numpy` way is still better. Now your only option is to trust this random notebook when it says that for many different applications GPUs have much lower execution times. Live long and prosper.

So far, the universal functions (`ufuncs`) we have encountered (whether executed on the CPU or GPU) take an array as input and return an identically shaped array of scalars by applying an operation element-wise. We now wish to generalize this concept to enable more complex behaviors, such as computing over specific sub-arrays and yielding output arrays with different dimensions. To achieve this, we use the `@guvectorize` decorator, as shown in the 2 examples below.



Now that we know how everything works, let's generate 10 million points, run the `numpy` and GPU (and even the plain Python implementation) functions and look at the difference in execution time.

```
In [200...] points = np.random.multivariate_normal([0,0], [[1.,0.9], [0.9,1.]], int(1e7)).astype(np.float32)
x = np.ascontiguousarray(points[:,0])
y = np.ascontiguousarray(points[:,1])
plt.figure(figsize=(7, 4), dpi=100)
_ = plt.hist(gpu_arctan2(y, x), bins=200)
```



```
In [201...] %timeit np.arctan2(y, x)
```

21.4 ms ± 2.18 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)

```
In [202...] %timeit gpu_arctan2(y, x)
```

44.3 ms ± 2.28 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

```
In [203...] # Plain Python
%timeit [math.atan2(point[1], point[0]) for point in points]
```

5.33 s ± 755 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

Note: LOL, this example was meant to show that the GPU is faster for more complex computations, but I think that Prof. Lamanna was maybe using a different GPU when he wrote this code, since the `numpy` way is still better. Now your only option is to trust this random notebook when it says that for many different applications GPUs have much lower execution times. Live long and prosper.

So far, the universal functions (`ufuncs`) we have encountered (whether executed on the CPU or GPU) take an array as input and return an identically shaped array of scalars by applying an operation element-wise. We now wish to generalize this concept to enable more complex behaviors, such as computing over specific sub-arrays and yielding output arrays with different dimensions. To achieve this, we use the `@guvectorize` decorator, as shown in the 2 examples below.

- Computing polar coordinates of random points:

```
In [204...] from numba import guvectorize

@guvectorize(['(float32[:], float32[:])'],
             '(n)->(n)',
             target='cuda')
def gpu_polar(vec, out):
    x = vec[0]
    y = vec[1]
```

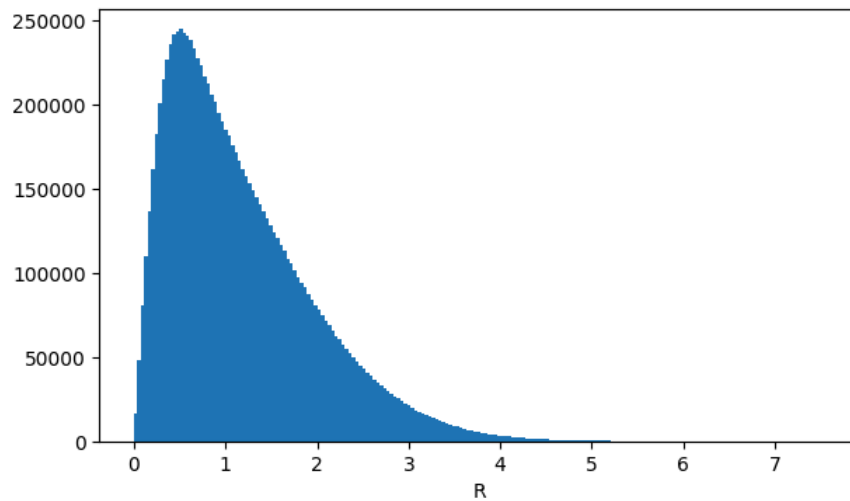
```

out[0] = math.sqrt(x**2 + y**2)
out[1] = math.atan2(y,x)

polar_coords = gpu_polar(points)

plt.figure(figsize=(7, 4), dpi=100)
_ = plt.hist(polar_coords[:,0], bins=200)
_ = plt.xlabel('R')

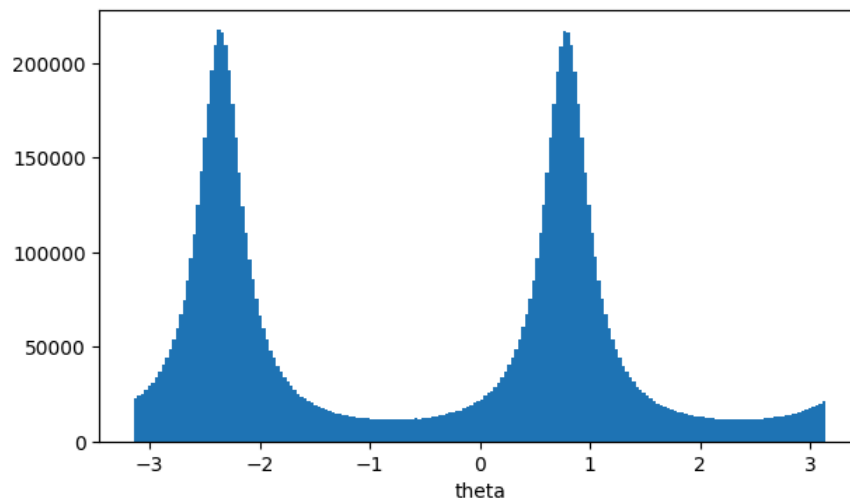
```



```

In [205... plt.figure(figsize=(7, 4), dpi=100)
_ = plt.hist(polar_coords[:,1], bins=200)
_ = plt.xlabel('theta')

```



- Computing the mean of the lines of a matrix:

```

In [206... @guvectorize(['(float32[:], float32[:])'],
              '(n)->()',
              target='cuda')

def gpu_average(array, out):
    acc = 0
    for val in array:
        acc += val
    out[0] = acc/len(array)
    print(len(array))

a = np.arange(100).reshape(20, 5).astype(np.float32)

gpu_average(a)

```

```

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will
likely result in GPU under-utilization due to low occupancy.
warn(errors.NumbaPerformanceWarning(msg))

```

```

Out[206... array([ 2.,  7., 12., 17., 22., 27., 32., 37., 42., 47., 52., 57., 62.,
        67., 72., 77., 82., 87., 92., 97.], dtype=float32)

```